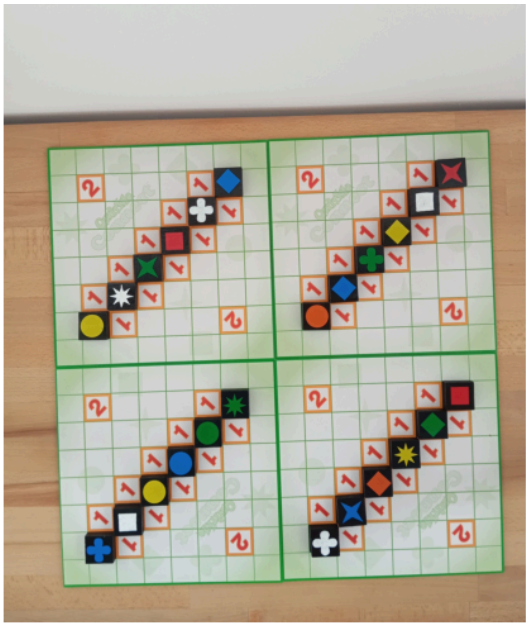
Project Report
Using OpenCV in order to automatically calculate the score in a game of Qwirkle Connect

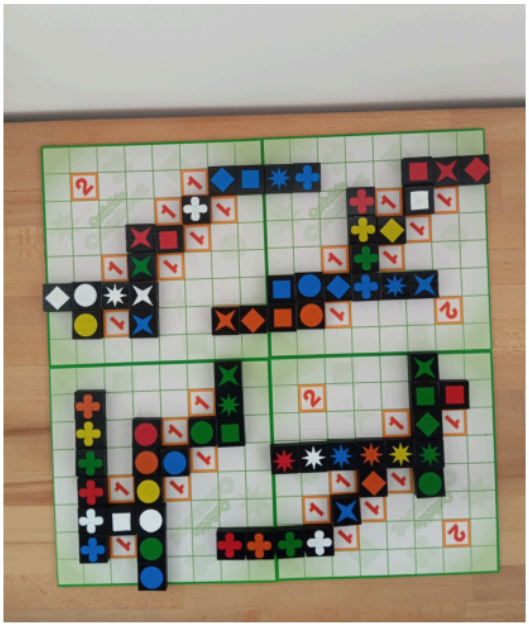
Introduction and Objectives:

This project is a homework assignment for the Computer Vision class (CAVA). The main objective is to apply various computer vision techniques learned in class to automatically extract visual information from a given set of plays in the game Qwirkle Connect.

The program needs to process images of the game board, detect the game state, and calculate the score. The image on the left depicts a game table at the start of a game, and the image on the right shows the same table at a random play state.



Initial table



Random play state

I chose to solve the problem by breaking it down into 4 distinct steps:

1. Detecting and extracting the game table to ensure a consistent environment.
2. Identifying the distinct places of the bonus points on the board.
3. Figuring out the pieces (shape and color) that have been placed on the table at a given turn.
4. Calculating the score of a turn based on the history of plays that have taken place.

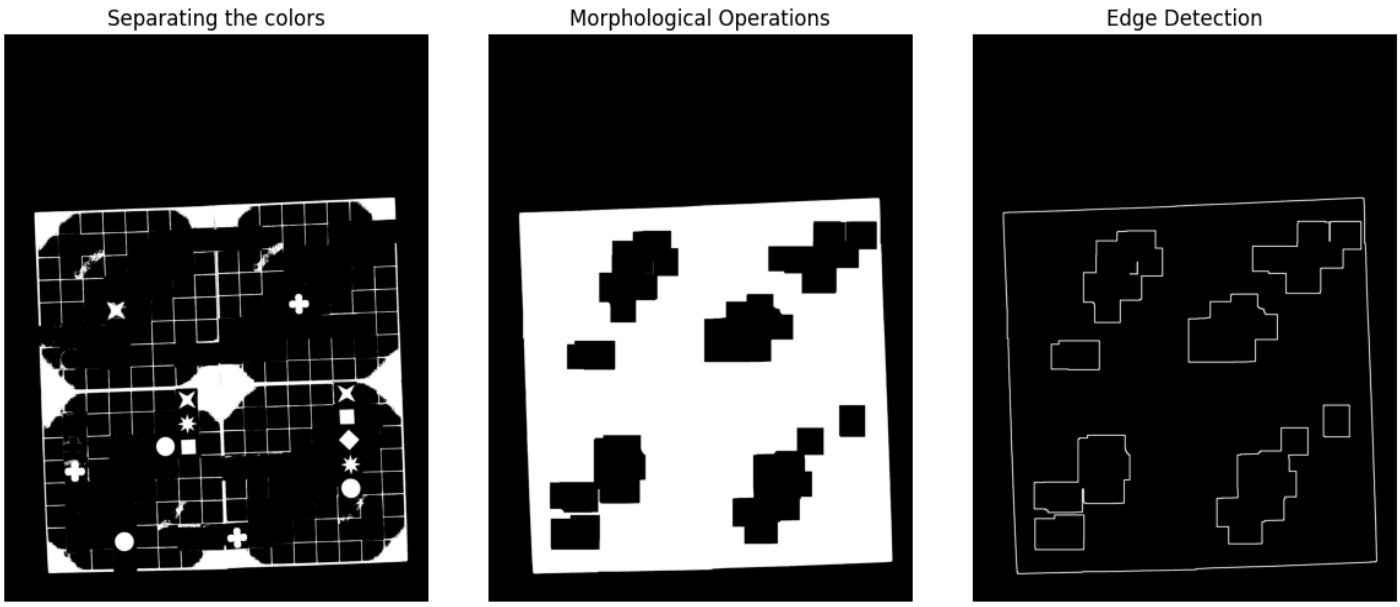
NOTE: Each one of the steps represents a different cell in the Jupyter Notebook.

Step 1: Detecting and extracting the game table.

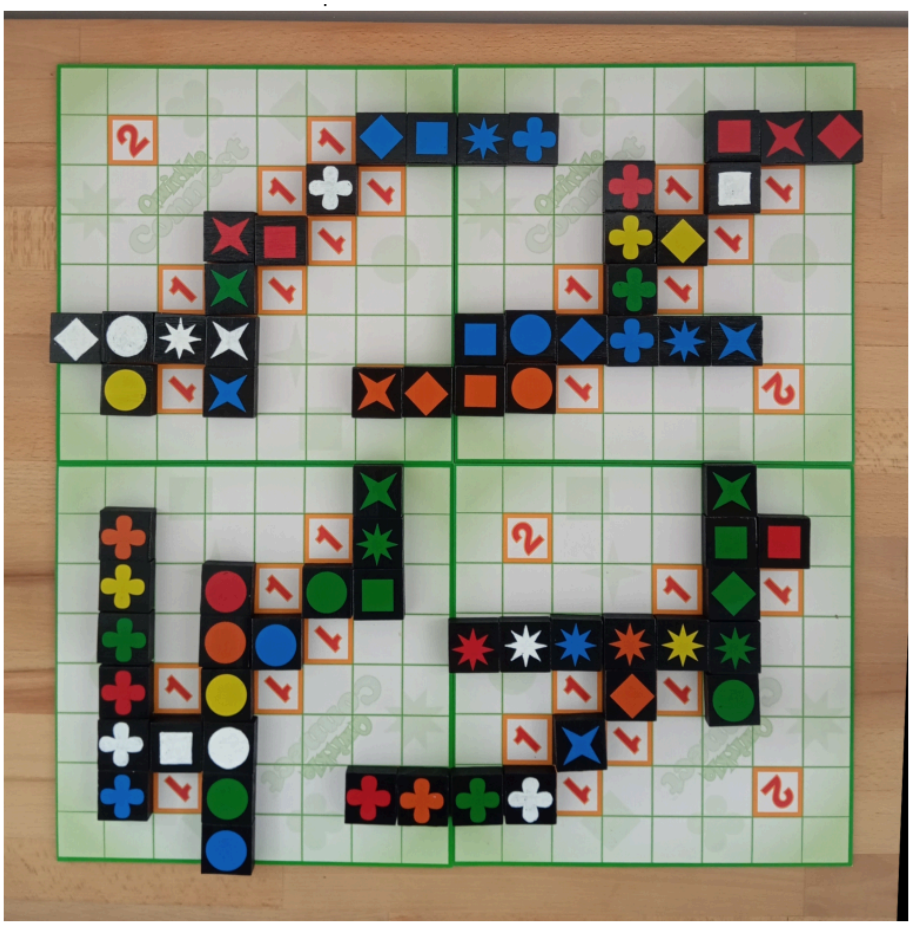
To analyze the game state accurately, we first need to 'enhance' the given images. Since the photos are taken with a phone, they may contain perspective distortions and background noise. The goal is to obtain a top-down view of the 16x16 grid, keeping padding in mind to ensure better accuracy for the template matching (so technically 18x18, 100px padding on all sides) in Step 3.

Implementation:

- **Separating the colors:** The image is converted from BGR to HSV color space. A color mask is created to isolate the green border of the board.
- **Morphological Operations:** To obtain a solid contour, I applied cv.morphologyEx with the cv.MORPH_CLOSE operation using a large kernel (60×60). This fills in any gaps in the green border caused by pieces overlapping the edge.
- **Edge Detection/Contours:** A Gaussian Blur is applied to reduce noise, followed by a Laplacian filter to highlight edges, I then converted the image to binary and applied dilate to get bigger edges. At the end I used cv.findContours to find the largest external contour.
- **Perspective Transform:** The four corners of the board are identified (cv.minAreaRect) and ordered (Top-Left, Top-Right, Bottom-Right, Bottom-Left). Using cv.getPerspectiveTransform and cv.warpPerspective, the image is warped into a fixed 1800×1800 resolution (including padding).



The steps that the image undergoes



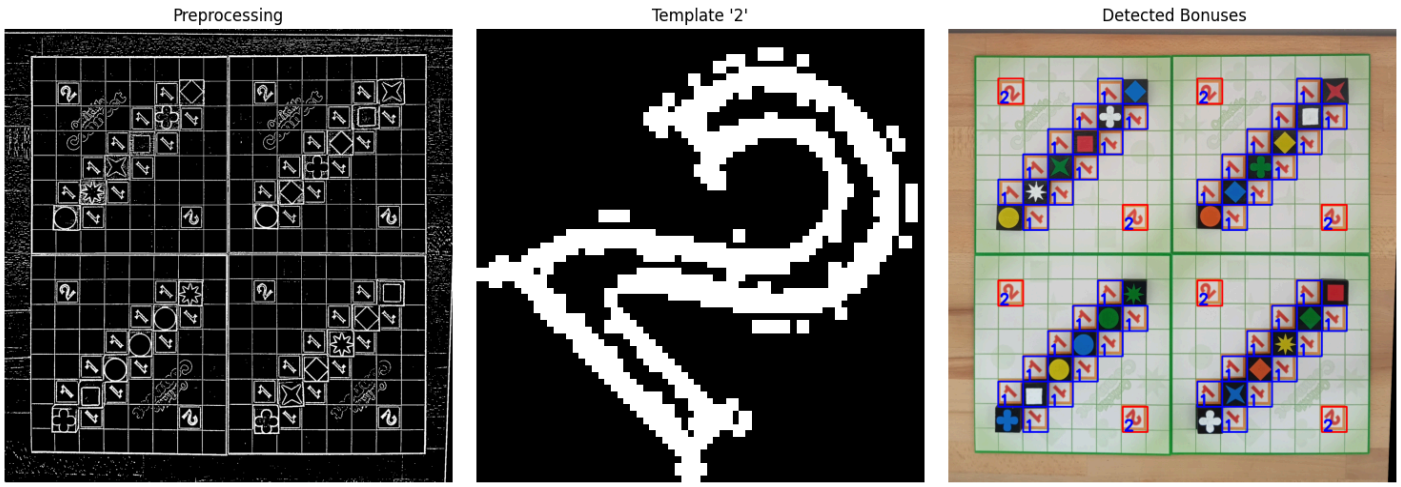
Final Result

Step 2: Identifying the distinct places of the bonus points on the board

In Qwirkle Connect, the board configuration can change at the start of each game. The bonuses are not always in the same spot. So we need a reliable way to determine their positions. My approach takes advantage of the fact that the first image of any game always consists of the board without any player pieces placed (except the starting ones). I analyze only this first image to save a matrix containing the bonus locations for the entire given game.

Implementation:

- **Preprocessing:** The extracted board image is converted to grayscale and then to a binary map using Adaptive Thresholding and morphological opening (cv.morphologyEx) to isolate text and symbols.
- **Template Matching for '2':** A cropped template of the number "2" (bonus_2.jpg-cropped, in the code, to remove the 15px padding) is matched against the board using cv.matchTemplate with cv.TM_CCOEFF_NORMED. The threshold is 0.7. I also rotated the bonus template by 90, 180, and 270 degrees to ensure all possible board configurations are detected.
- **Finding the '1' Bonuses:** Since the board consists of four identical 8×8 quadrants rotated, detecting the "2" cells allows us to also find the orientation of each quadrant. Based on the position of the "2", the algorithm automatically places the "1" bonuses along the specific diagonals (ascending or descending) defined in the code by simply checking the positions of the ‘2’ in the matrix.



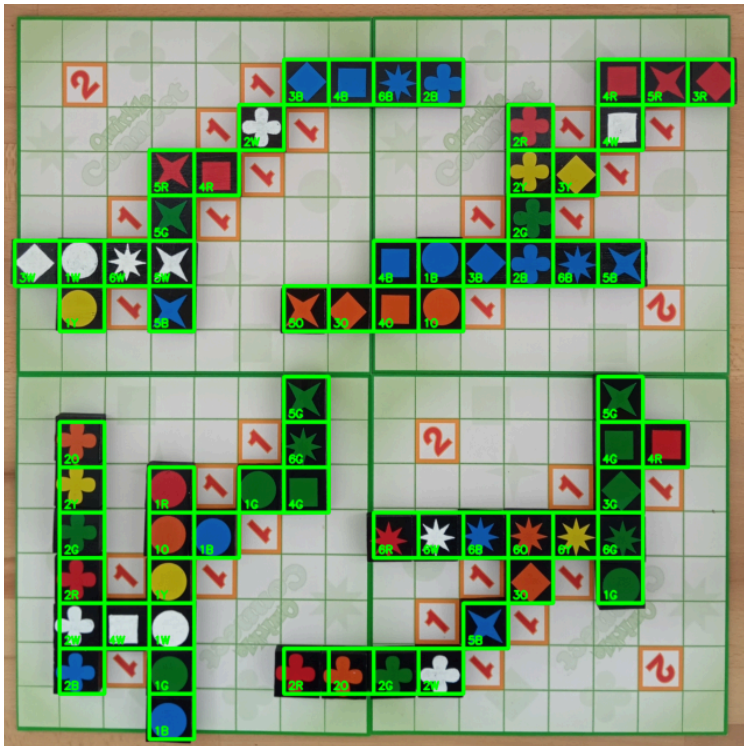
Step 3: Figuring out the pieces placed on the table

The most important step in the entire problem is identifying the placement, type, and color of the game pieces.

Shape: I converted the image to grayscale and applied a Gaussian Blur to eliminate noise. With the image in this format, I applied Template Matching using all 6 piece templates, converted in grayscale. To get high accuracy, I implemented two different tricks:

1. Rotation: I rotated the templates to check all orientations (0, 90, 180, 270)
2. Scaling: I scaled the templates from 90% to 110% in 5% intervals to ensure I detect pieces even if their size varies slightly in the photo.

Color Classification: To determine the color of the pieces, I created a copy of the starting image and converted it to the HSV color space, which offers a more robust approach to color detection. I iterated through every cell of the game board, extracted a 10×10 pixel region from the center of the cell, and calculated the average Hue and Saturation. These values were checked against a set of hardcoded ranges for each color. For white pieces, I relied on Saturation, for red pieces, I took into consideration the circular nature of the HSV hue cone.



Step 4: Calculating the score of a turn based on the history of plays

The final step is to implement the game logic. The code maintains a previous_matrix and compares it with the current_matrix.

Algorithm:

- Identify New Pieces: Cells that were empty in the previous turn but are occupied now are flagged as new_pieces.
- Line Checking: For each new piece, the algorithm scans horizontally and vertically (using the function get_full_line_score) to find lines of pieces.
- Scoring Rules:
    - Base Score: 1 point per piece in the line.
    - Qwirkle Bonus: +6 points if the line length reaches 6.
    - Board Bonus: If a piece sits on a bonus tile detected in Step 2, its value is added to the score.
- Avoid Duplicates: A unique ID is generated for every scored line (ex: H_row_col) to ensure a line is not counted multiple times in the same turn.

```
Start checking game 1:
Checking the initial configuration: 1_00.jpg
Checking quadrants for bonuses.
Checking move: 1_01.jpg
    Saved: 2H:4B, 2I:6B, 2J:2B | Score: 6
Checking move: 1_02.jpg
    Saved: 12B:2G, 13B:2R, 14B:2W | Score: 7
Checking move: 1_03.jpg
    Saved: 4D:5R, 6D:5W, 7D:5B | Score: 10
Checking move: 1_04.jpg
    Saved: 14D:1W, 15D:1G, 16D:1B | Score: 8
Checking move: 1_05.jpg
    Saved: 6A:3W, 6B:1W | Score: 7
Checking move: 1_06.jpg
    Saved: 11D:1R, 12D:1O | Score: 15
Checking move: 1_07.jpg
    Saved: 3L:2R, 4L:2Y, 6L:2B | Score: 10
Checking move: 1_08.jpg
    Saved: 15G:2R, 15H:2O, 15I:2G | Score: 6
```

Output cell during runtime, with scores + plays